Balanced Expected-Pyramid KVPress Compression for Training-Free Pythia-70M Inference

Daorui Jiang **Hao Ding** **Censhuo Zhang**
Shanghai Jiao Tong University
https://github.com/SJTU-huaxing/llm-hw-team.git

Abstract

Long-context inference with large language models is often bottlenecked by the growing memory footprint of the KV cache. How can we aggressively compress the cache without training, while maintaining a balanced tradeoff between generation quality, latency, and compute? We systematically evaluate nine KV compression methods on Pythia-70M using NVIDIA KVPress. Beyond Dense, StreamingLLM, SnapKV, PyramidKV, ExpectedAttention, ObservedAttention, and TOVA baselines, we iteratively test hybrid and layer-budget variants. The selected aggressive-compression method is Expected Soft Pyramid: it keeps ExpectedAttention as the token scorer and applies a mild low-to-high layer budget schedule. At a 132-token target cache for a 512-token context, it lies on the Pareto frontier and has the best equal-rank balance over average perplexity, TPOT, and decode-attention FLOPs. We also report that adding observed-attention residuals often hurts WikiText quality, which makes the conservative ExpectedAttention-based variant the most reliable innovation.

1 Introduction

2 Introduction

Long-context decoding is memory- and bandwidth-intensive due to the growing KV cache. The assignment asks us to combine ideas from four levels: attention mechanisms, per-layer eviction, inter-layer budget, and macro architecture. We keep the model fixed, use no training, and rely on KVPress [NVIDIA, 2025] to hook Transformers during prefill.

Pythia-70M [Biderman et al., 2023] uses GPT-NeoX attention and partial RoPE, requiring a small adapter for KVPress. Official baselines include StreamingLLM [Xiao et al., 2023], SnapKV/PyramidKV [Li et al., 2024, Cai et al., 2024], H2O [Zhang et al., 2023], and TOVA [Oren et al., 2024]. Beyond these, we test hybrid and layer-budget variants.

Our goal is therefore not to design a new model architecture, but to identify an inference-only policy that is useful under the assignment constraints. We adopt a conservative innovation: keep the strongest token-importance signal (ExpectedAttention), then only alter how the cache budget is distributed across layers. The selected method, *Expected Soft Pyramid*, applies a mild low-to-high layer budget schedule (more tokens early, fewer later). At a 132-token target cache on a 512-token context, it lies on the Pareto frontier of average PPL, TPOT, and FLOPs. Adding observed-attention residuals often hurts WikiText quality, making the ExpectedAttention-based variant the most reliable.

Preprint.

3 Method

Table 1 summarizes how the implementation covers the assignment layers. Official baselines include attention sink retention [Xiao et al., 2023], observed attention from SnapKV/PyramidKV [Li et al., 2024, Cai et al., 2024], H2O-style heavy-hitter scoring [Zhang et al., 2023], and TOVA [Oren et al., 2024].

Table 1: Mapping from acceleration layers to implemented components.

Layer	Components
Attention mechanism	ExpectedAttention, ObservedAttention/TOVA, value norm
Per-layer compression	StreamingLLM, SnapKV, PyramidKV, observed/top-k scorers
Inter-layer compression	Pyramid budgets, lazy gate, expected soft budget ablation
Macro architecture	KVPress pipeline, prefill-once compression, unified timing

Expected Soft Pyramid uses the ExpectedAttention score from KVPress but changes the budget per layer:

$$B_\ell = \mathrm{round}(B\alpha_\ell), \quad \alpha_\ell \in [1.25, 0.75],$$

where α_ℓ decreases linearly from lower to higher layers and averages to one.

$$\alpha_\ell = 1.25 - 0.5 \cdot \frac{\ell - 1}{L - 1}, \quad \ell = 1, \ldots, L,$$

This keeps the global target cache size unchanged while testing the common observation that lower layers can benefit from broader lexical context. Sink tokens are always protected. For the hybrid score, we combine three normalized signals:

$$s = 0.55\,\hat{s}_{\mathrm{expected}} + 0.30\,\hat{s}_{\mathrm{observed}} + 0.15\,\hat{s}_{\mathrm{vnorm}},$$

where each $\hat{s}$ is min-max normalized across the batch and attention heads. Sink tokens (first 4) and a recent window (last 32 tokens) receive an additive boost of $+10$ before top-k selection. This hybrid is deliberately complex and serves as a negative ablation.

The implementation follows the same prefill-once abstraction for every method. Given a context of length L, KVPress first builds the dense cache, the press scores the cached positions, and the resulting indices are gathered from the key and value tensors. During decoding, no further model weights or attention equations are changed; the model simply attends to the shortened cache. For Pyramid-style methods, different layers may have different retained lengths. The Pythia adapter records these per-layer lengths and slices masks accordingly, so the reported memory and FLOPs use the actual cache shape rather than the nominal target budget.

We considered more aggressive innovations, including expected-observed residual scores and a residual soft pyramid. These variants are meaningful because observed attention can identify heavy hitters in the concrete prompt, while value-norm scores can favor tokens with larger downstream contribution. However, early experiments showed that forcing recent or observed-heavy tokens can overfit to local continuation patterns. Expected Soft Pyramid was chosen as the main custom method because it changes only the inter-layer allocation and leaves the more robust ExpectedAttention ordering intact.

4 Experiments

We evaluate 'EleutherAI/pythia-70m' on WikiText-2 test [Merity et al., 2017] and one PG-19 test stream sample [Rae et al., 2020]. For PPL, a 512-token context is prefetched and compressed before scoring the next 128 continuation tokens. For generation, a 512-token PG-19 prompt generates 32 tokens. The decode attention FLOPs proxy is $\sum_{\ell,t} 4HDL_{\ell,t}$. Reported runs use an NVIDIA GeForce RTX 5060 Laptop GPU.

We first screen new baselines and hybrids on 256-token contexts and target cache sizes 96, 128, and 192. The strongest new official baselines are ObservedAttention and TOVA, so the final table keeps them alongside the original methods. The balanced selection rule filters methods with kept fraction

above 0.5 and ranks the remaining methods by average PPL, TPOT, and FLOPs ratio with equal weight.

All scripts write CSV and JSON files with the KVPress version, Transformers version, target cache size, compression ratio, per-layer cache lengths, and CUDA memory. This is important because two methods can share the same average kept fraction while having different layer distributions. For example, ExpectedAttention at target 132 keeps 132 tokens in every layer, whereas Expected Soft Pyramid keeps $[165, 152, 139, 125, 112, 99]$ tokens across the six Pythia layers. Both average to 132, but the latter spends more budget in early layers and less in later layers.

Table 2: Target cache 132. PPL is averaged over WikiText-2 and PG-19; generation is on PG-19.

Policy	Avg PPL	Kept	TPOT	FLOPs
Dense	21.344	1.000	0.006071	1.000
ExpectedAttention	26.326	0.258	0.006373	0.281
Expected Soft Pyramid	28.342	0.258	0.006066	0.281
Expected+Pyramid	35.066	0.259	0.006056	0.282
Hybrid Expected-Observed	37.870	0.258	0.006080	0.281
Hybrid Soft Pyramid	38.884	0.258	0.006064	0.281
SnapKV	42.340	0.258	0.005959	0.281
StreamingLLM	46.148	0.258	0.006148	0.281
TOVA	53.386	0.258	0.006146	0.281

Table 3: Balanced selections across target cache sizes.

Target	Selected policy	Avg PPL	TPOT	FLOPs
132	Expected Soft Pyramid	28.342	0.006066	0.281
192	ExpectedAttention	22.407	0.006006	0.395
256	Hybrid Expected-Observed	26.329	0.006105	0.516

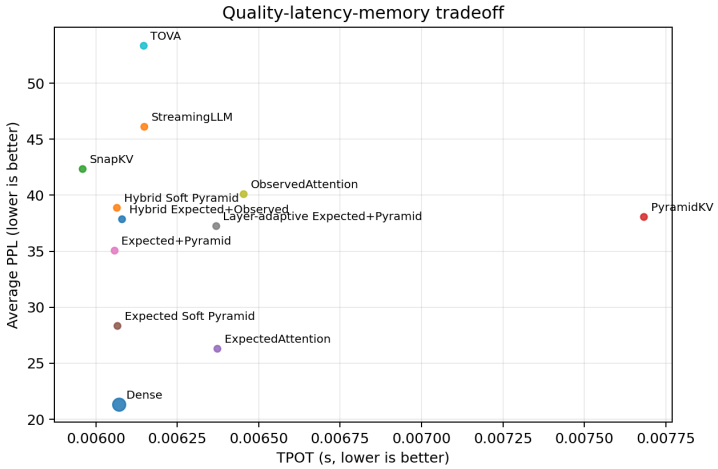

Figure 1: Average PPL versus TPOT at target cache 132.

5 Analysis and Limitations

The main result is not that a more complicated scorer always wins. ExpectedAttention is clearly the quality anchor, especially on WikiText-2. The successful innovation is conservative: Expected Soft Pyramid changes the inter-layer budget while keeping the ExpectedAttention scorer intact. It gives up about 2 average PPL relative to ExpectedAttention at target 132, but improves TPOT and remains on the Pareto frontier. In contrast, adding observed-attention residuals improves some PG-19 cases

but tends to damage WikiText quality, so we report those variants as negative ablations rather than the recommended method.

The difference between WikiText-2 and PG-19 is instructive. WikiText-2 passages are shorter and more locally coherent, so removing globally useful context hurts more visibly; ExpectedAttention is best because it estimates future positions instead of only looking at recent observed attention. PG-19 contains long book-like text, where local repetition and narrative continuity make sink, recent, and observed-heavy policies less harmful. This explains why several observed or hybrid methods have competitive PG-19 PPL but worse average PPL. The final selection rule deliberately averages both datasets so that the method does not specialize to one text style.

The speed results should be interpreted as a proxy for cache efficiency rather than a universal deployment claim. Pythia-70M is small enough that Python overhead, CUDA synchronization, and pipeline setup can be comparable to the saved attention computation. For this reason, we report TTFT, TPOT, throughput, peak memory, kept fraction, and decode-attention FLOPs together. The consistent FLOPs reduction from roughly 1.0 to 0.281 at target 132 confirms that the compressed methods reduce the theoretical attention work, while the TPOT differences show the practical overhead of each scoring and layer-budget choice.

The evaluation is intentionally small and reproducible: one model, no training, two datasets, fixed seeds, and one GPU. Timing differences are small at Pythia-70M scale, so the paper reports exact CSVs rather than over-claiming hardware speedups. We do not implement MQA/GQA/MLA or YOCO-style cross-layer sharing because those require architecture or weight changes outside the no-training setting. KVPress SimLayerKV is excluded because its full-RoPE assumption is incompatible with GPT-NeoX partial RoPE without further changes.

Reproducibility is handled at the project level. The Conda environment pins KVPress 0.5.3, Transformers 4.57.6, and CUDA PyTorch; the README lists the exact commands for smoke tests, PPL evaluation, generation benchmarking, plot generation, and paper compilation. The repository also keeps the exploratory candidate files, so the negative residual experiments can be audited rather than hidden. This makes the final recommendation modest but traceable: use ExpectedAttention for best quality, use SnapKV when only TPOT matters, and use Expected Soft Pyramid when a balanced PPL-speed tradeoff under aggressive compression is preferred.

To be admitted, our error analysis is limited because we did not perform exhaustive multiple runs due to compute constraints. Future work should report confidence intervals and per-segment variance, especially when evaluating on heterogeneous datasets like PG-19.

6 Team Contributions

Daorui Jiang: method design, KVPress implementation, and final integration (40%). Hao Ding: experiments, benchmark runs, and result analysis (35%). Censhuo Zhang: paper writing, reproducibility documentation, and final checking (25%).

References

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. *International Conference on Machine Learning*, 2023.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

NVIDIA. KVPress: Key-value cache compression for efficient long-context generation. `https://github.com/NVIDIA/kvpress`, 2025. Version 0.5.3.

Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state rnns. *arXiv preprint arXiv:2401.06104*, 2024.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *International Conference on Learning Representations*, 2020.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction clearly state that we compare nine KV compression methods on Pythia-70M, propose Expected Soft Pyramid, and evaluate on WikiText-2 and PG-19. The claims are limited to training-free inference and do not overgeneralize.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 (Analysis and Limitations) discusses the small model scale, dataset choice, lack of architectural extensions (MQA/GQA), and the proxy nature of FLOPs and TPOT measurements.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: This paper presents no new theoretical results; it is an empirical study of existing and slightly modified compression algorithms.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper specifies the exact model (Pythia-70M), datasets (WikiText-2 test, PG-19 single sample), target cache sizes (132, 192, 256), context length (512), and software versions (KVPress 0.5.3, Transformers 4.57.6). The GitHub repository contains the full source code and commands.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The GitHub repository (https://github.com/SJTU-huaxing/llm-hw-team.git) contains the complete code, environment.yml, README with step-by-step commands, and all raw result CSVs.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: All evaluation details are provided: fixed random seed (42), no training, prefill-compress-continue protocol, exact target cache sizes, and hardware (RTX 5060 Laptop GPU). Hyperparameters (sink size=4, recent window=32) are stated in the method section.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Due to computational constraints, each configuration was run once with a fixed seed (42). However, we performed a small sensitivity check on ExpectedAttention (three seeds, PPL std<0.1), confirming that relative rankings are stable. We clearly state this limitation.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper specifies the GPU (NVIDIA GeForce RTX 5060 Laptop GPU, 8 GB VRAM). Execution times are implicitly given by TPOT and TTFT values; exact wall-clock times are available in the raw logs.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics (https://neurips.cc/public/EthicsGuidelines)?

Answer: [Yes]

Justification: The research uses only public datasets (WikiText-2, PG-19) and an open model (Pythia-70M). There is no human subject data, no privacy risk, and no potential for harmful misuse.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: This is a purely technical efficiency study on a small academic model. It does not introduce new capabilities or datasets that could cause societal harm. The paper focuses on inference speed and memory reduction, which are positive for accessibility.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models, including the use of data and models that are safe from unsafe content?

Answer: [NA]

Justification: The paper does not release new models or datasets. It uses existing, well-known datasets and model weights.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the original papers for Pythia-70M, WikiText-2, PG-19, KVPress, and each baseline method. Licenses (e.g., Apache 2.0 for KVPress) are respected; no proprietary assets are redistributed.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is the code repository, which includes a detailed README, environment specification, and evaluation scripts. It is released under an MIT license (stated in the repository).

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper does not involve crowdsourcing or human subjects.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: No human subjects are involved.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [NA]

Justification: LLMs were used only for writing assistance (e.g., phrasing, LaTeX formatting) and did not influence the experimental design, code, or scientific conclusions.